

TRABAJO PRÁCTICO INTEGRADOR

SpamScanner 2.0

Modelos Formales Aplicados a la Detección de Spam en SMS

Materia	Autómatas y Gramáticas
Año	3° Año — Ingeniería en Informática
Ciclo Lectivo	2026
Modalidad	Grupal (2 a 3 integrantes)
Formato de entrega	Código fuente + Informe técnico PDF + Presentación oral

1. DESCRIPCIÓN GENERAL

El Trabajo Práctico Integrador tiene como propósito que los alumnos construyan un sistema completo de análisis y clasificación de mensajes SMS, aplicando en forma articulada todos los modelos formales estudiados durante la materia. El sistema procesa mensajes del SMS Spam Collection Dataset y los transforma en una decisión de clasificación (spam / ham) a través de cuatro etapas que recorren la Jerarquía de Chomsky de extremo a extremo.

Cada etapa del pipeline corresponde a un modelo formal distinto, y la salida de cada una es la entrada de la siguiente. Esto permite experimentar, en un contexto real y motivador, cómo los diferentes niveles de expresividad formal se complementan para resolver un problema computacional concreto.

Pipeline del sistema

Texto crudo SMS

- ↓ Etapa 1 — Máquina de Turing → Normalización del texto
- ↓ Etapa 2 — Expresiones Regulares → Tokenización
- ↓ Etapa 3 — Heurística de pesos → Clasificación spam/ham
- ↓ Etapa 4 — Gramática Libre de Contexto → Validación estructural
- ↓ Veredicto final: HAM / SPAM / SPAM ATÍPICO

2. CONTENIDOS CONCEPTUALES

El trabajo integra y pone en práctica los siguientes contenidos abordados a lo largo de la materia:

ETAPA 1 · Máquina de Turing

Contenidos movilizados:

- Definición formal de MT: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$
- Función de transición $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- Traza de ejecución / Descripción Instantánea (ID)
- Diagrama de estados de la MT
- MT como transformador de cadenas (no solo aceptador)

ETAPA 2 · Expresiones Regulares

Contenidos movilizados:

- Expresiones regulares: operadores básicos (concatenación, unión, clausura)
- Equivalencia entre ER, AFD y Gramáticas Regulares (Tipo 3)
- Diseño de patrones para reconocimiento de tokens (MONEY, URL, PHONE, CAPS, WORD)
- Implementación en Python con el módulo `re`
- Análisis léxico como caso especial de reconocimiento por autómata finito

ETAPA 3 · Clasificación Heurística

Contenidos movilizados:

- Evaluación empírica de modelos formales: accuracy, precisión, recall
- Relación entre expresividad del modelo y capacidad de clasificación
- Experimentación con parámetros (umbral U) y análisis de resultados
- Pensamiento computacional: toma de decisiones basada en evidencia formal

ETAPA 4 · Gramáticas Libres de Contexto

Contenidos movilizados:

- Definición formal de GLC: $G = (V, T, P, S)$
- Derivaciones y árboles sintácticos de derivación

- Ambigüedad en gramáticas y técnicas de desambiguación
- Diseño de gramáticas para modelar estructura de lenguaje natural restringido
- Relación entre GLC, Autómatas de Pila y algoritmos de parsing (CYK, LL, LR)
- Posición de las GLC en la Jerarquía de Chomsky (Tipo 2)

3. MODALIDAD DE TRABAJO

3.1 Organización grupal

El trabajo es de carácter grupal. Cada equipo estará compuesto por 2 a 3 integrantes.

3.2 Entregables

Entregable	Descripción y formato
Código fuente	Repositorio Python. Debe incluir un script principal que ejecute el pipeline completo y produzca las salidas de cada etapa. Se permite usar la librería <code>turing_machine</code> o implementación propia.
Informe técnico (PDF)	Documento formal que incluye: definición y diagrama de estados de la MT, traza de ejecución, patrones de ER utilizados, tabla de accuracy para distintos valores de U, especificación de la GLC, árboles de derivación, análisis de ambigüedad y conclusión crítica.
Presentación oral	Exposición en clase de 4 a 5 diapositivas por grupo. Debe cubrir: funcionamiento del sistema, decisiones de diseño más relevantes y lecciones aprendidas.

3.3 Dataset y requisito de datos

El sistema debe procesar al menos 100 mensajes del SMS Spam Collection Dataset (50 ham / 50 spam) para la generación de métricas. Los análisis profundos (trazas, árboles de derivación, análisis de ambigüedad) pueden realizarse sobre casos puntuales seleccionados por el grupo.

Los grupos son libres de incorporar mensajes adicionales del dataset para enriquecer el análisis, siempre que se respete el balance mínimo indicado.

3.4 Herramientas

- Lenguaje de implementación: Python
- Módulos permitidos: re (expresiones regulares), turing_machine (o implementación propia de MT), cualquier librería estándar para manejo de archivos y métricas
- Para la documentación: cualquier procesador de texto o herramienta de diagramación.
- El uso de herramientas de IA generativa está permitido para asistencia, pero el grupo debe poder explicar y defender cada decisión de diseño en la presentación oral

4.1 Objetivo general

Que los alumnos sean capaces de diseñar, implementar y evaluar un sistema de procesamiento de lenguaje natural basado en modelos formales, integrando en una solución cohesionada los cuatro niveles de la Jerarquía de Chomsky estudiados durante la materia.

4.2 Objetivos específicos

Etapla 1 — Máquina de Turing

- Diseñar formalmente una MT que resuelva un problema de transformación de cadenas, especificando todos sus componentes: $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$.
- Construir el diagrama de estados completo de la MT diseñada.
- Trazar la ejecución de la MT sobre una cadena concreta, justificando cada transición.
- Comprender la MT no solo como aceptador de lenguajes sino como transformador de cadenas (preprocesador de datos).

Etapla 2 — Expresiones Regulares

- Diseñar expresiones regulares que reconozcan patrones lingüísticos específicos (cifras monetarias, URLs, números de teléfono, palabras en mayúsculas).
- Implementar un tokenizador en Python utilizando el módulo re.
- Justificar por qué los patrones a reconocer son lenguajes regulares y no requieren mayor expresividad.

Etapla 3 — Clasificación y Evaluación

- Aplicar un modelo heurístico basado en pesos de tokens para clasificar mensajes.
- Experimentar con distintos valores del umbral U y analizar su impacto en la exactitud del clasificador.
- Desarrollar pensamiento crítico sobre la relación entre complejidad del modelo formal y su efectividad práctica.

Etapas 4 — Gramáticas Libres de Contexto

- Diseñar una GLC que modele la estructura semántica del spam, con producción de al menos tres componentes obligatorios (gancho, contenido, cierre).
- Construir árboles de derivación para cadenas aceptadas por la gramática.
- Analizar si la gramática diseñada es ambigua y, en caso afirmativo, proponer una versión equivalente no ambigua.
- Distinguir entre mensajes clasificados como spam por puntaje y mensajes que además cumplen una estructura formal (spam canónico vs. spam atípico).

Objetivo transversal — Comunicación técnica

- Redactar un informe técnico formal que documente las decisiones de diseño, los resultados obtenidos y las conclusiones del análisis.
- Exponer oralmente el trabajo realizado ante la clase, con capacidad de responder preguntas sobre el diseño y la implementación.

5. RESULTADOS DE APRENDIZAJE ESPERADOS

Al finalizar este trabajo práctico, se espera que cada integrante del grupo sea capaz de demostrar los siguientes resultados de aprendizaje:

RA 1 · Modelado con Máquinas de Turing

- Especifica formalmente una MT completa para un problema de transformación de texto, incluyendo todos sus componentes.
- Construye el diagrama de estados correspondiente y verifica su corrección.
- Traza paso a paso la ejecución de la MT sobre una entrada concreta, identificando cada transición aplicada.
- Explica la diferencia entre una MT usada como aceptador y una usada como transformador.

RA 2 · Diseño e implementación de Expresiones Regulares

- Diseña expresiones regulares precisas para reconocer patrones textuales complejos.
- Implementa un tokenizador funcional en Python y lo integra en un pipeline mayor.
- Argumenta por qué los patrones de tokenización corresponden a lenguajes regulares y no requieren mayor expresividad.

RA 3 · Evaluación experimental de modelos formales

- Aplica el clasificador heurístico sobre un conjunto de datos real y mide su nivel de aciertos.
- Experimenta con distintos valores del umbral U y analiza cuantitativamente el impacto de cada uno.

- Elabora conclusiones fundadas sobre qué componente del sistema aportó mayor poder discriminativo.

RA 4 · Diseño y análisis de Gramáticas Libres de Contexto

- Diseña una GLC que captura la estructura semántica del spam con al menos tres componentes diferenciados.
- Construye correctamente árboles de derivación para tres mensajes aceptados por la gramática.
- Detecta y resuelve ambigüedad en la gramática, proponiendo una versión equivalente no ambigua.
- Distingue entre mensajes spam canónicos y spam atípico, fundamentando la diferencia desde el modelo formal.

RA 5 · Integración y pensamiento sistémico

- Articula coherentemente los cuatro modelos formales en un pipeline funcional donde cada etapa alimenta a la siguiente.
- Relaciona cada etapa del sistema con el nivel correspondiente de la Jerarquía de Chomsky.
- Identifica las limitaciones de cada modelo formal y justifica por qué la combinación de modelos supera a cualquiera aplicado en forma aislada.

RA 6 · Comunicación técnica

- Redacta un informe técnico claro, organizado y con el nivel de formalismo adecuado para una audiencia especializada.
- Expone oralmente el trabajo con dominio del tema, claridad conceptual y capacidad de responder preguntas sobre las decisiones de diseño tomadas.

6. DESARROLLO DEL SISTEMA

6.1 Descripción General

El objetivo es construir un sistema de análisis y clasificación de SMS utilizando modelos formales. El pipeline procesará mensajes del **SMS Spam Collection Dataset**, transformando el texto crudo en una decisión de clasificación a través de cuatro etapas que recorren la Jerarquía de Chomsky.

Requisito de datos: Procesar al menos 100 mensajes (50 ham / 50 spam) para las métricas, y realizar análisis profundos sobre casos puntuales.

6.2 Desarrollo del Sistema

Etapas 1 — Normalización Selectiva (Máquina de Turing)

Para que el análisis sea efectivo, debemos limpiar el ruido sin perder información clave.

- **La Tarea:** Implementar una Máquina de Turing (MT) que actúe como un filtro de caracteres.
- **Comportamiento:** La MT debe conservar:
 - letras (A-Z, a-z),
 - dígitos (0-9),
 - espacios y
 - **símbolos críticos para la detección:**
 - \$
 - . (punto)
 - :
 - /
- Cualquier otro símbolo (como !, ¿, *, (,)) debe ser reemplazado por un espacio.
- **Entregable técnico:** Definir formalmente la MT: $(Q, \Sigma, \delta, q_0, F)$.
- **Traza:** Mostrar la traza de ejecución completa para un mensaje de **no más de 20 caracteres** (ejemplo: "WIN \$1000 now!").

Etapas 2 — Tokenización con Expresiones Regulares

Sobre el texto normalizado por la MT, se utilizará el módulo "re" de Python para transformar la cadena de caracteres en una **secuencia de tokens**.

- **Tokens a identificar:**
 - MONEY: Cifras precedidas o terminadas por \$, £ o €.
 - URL: Patrones que sigan la estructura de un enlace (ej. www.x.com o http://...).
 - PHONE: Secuencias numéricas de 7 o más dígitos.
 - CAPS: Palabras de 3 o más letras que estén completamente en mayúsculas.

- **WORD:** Cualquier otra palabra que no entre en las categorías anteriores.
- **Salida:** Una lista ordenada de tipos de tokens. Ejemplo: WORD, CAPS, MONEY, URL.

Etapas 3 — Clasificación por Peso Predictivo

En esta etapa, el sistema realiza una clasificación heurística basada en la densidad de "señales de spam".

- **Lógica:** Asignar pesos a los tokens: MONEY (3), PHONE (3), URL (2), CAPS (1), WORD (0).
- **Proceso:** Sumar los pesos de un mensaje. Si la suma supera un **Umbral U**, se etiqueta como **SPAM**, de lo contrario es **HAM**.
- **Evaluación:** Los alumnos deben probar al menos 3 valores diferentes para **U** sobre los 100 mensajes y reportar cuál ofrece la mejor precisión (Accuracy) comparando con la etiqueta real del dataset.

Etapas 4 — Validación Estructural (Gramáticas Libres de Contexto)

Los mensajes que fueron etiquetados como **SPAM** en la Etapa 3 pasan por un último filtro de validación estructural. Aquí verificamos si el spam "está bien construido" según un modelo formal.

- **Reducción de Alfabeto:** Para la gramática, el alfabeto terminal será (caps, money, contact, text) donde:
 1. contact unifica los tokens PHONE y URL.
 2. text representa uno o más tokens WORD consecutivos.
- **Diseño de la GLC:** Diseñar una gramática que modele la "semántica" del spam. Una estructura válida debe tener obligatoriamente un componente de **GANCHO** (caps), un **CONTENIDO** (money o text) y un **CIERRE** (contact).
- **Desafíos de la Etapa:**
 1. **Árboles:** Dibujar el árbol de derivación para 3 mensajes que sí cumplen la gramática.
 2. **Ambigüedad:** Identificar si la gramática permite que un mismo mensaje tenga dos estructuras distintas. Si es ambigua, proponer una versión equivalente que no lo sea.
 3. **Veredicto Final:** Si un mensaje era SPAM por puntaje pero no es aceptado por la GLC, se marca como "Spam Atípico" y se analiza su caso en el informe.

6.3 Entregables

1. **Código Fuente:** Repositorio en Python. Se recomienda el uso de librerías como turing_machine (o una implementación propia) y re.
2. **Informe Técnico (PDF):**
 - Diagrama de estados de la MT.
 - Tabla de resultados de precisión variando el umbral U.
 - Especificación de la GLC.
 - Análisis de ambigüedad y árboles sintácticos.
3. **Análisis Crítico:** Una breve conclusión sobre qué etapa fue más efectiva para detectar mensajes engañosos y por qué.
4. Presentación en clase, cada grupo deberá exponer en clase con una presentación de 4/5 diapositivas como realizó el trabajo, cómo funciona y detalle de implementación.